Judge vs. React: dissociating perception from timing in VLM reflex control

Josh Simnitt
jsimnitt@gmail.com

2026

Abstract

A vision-language-model (VLM) agent that must time a single action in a first-person scene can fail two ways that existing game benchmarks cannot pull apart: it can misjudge the scene (perception), or judge correctly but act too late (timing). We build a minimal, deterministic first-person pit-jump task that reduces an episode to one decision, labels each decision in- or out-of-window from the sim's own dynamics, and injects a control delay D so that lateness varies independently of judgment. In the dimensionless coordinate $r = vD/W$ (distance travelled during the delay over window width), a policy that fires at the window's leading edge clears iff $r < 1$ by construction; oracle sweeps on the pit ($r^\star = 1.17$, bracketed by the delay grid between 1.04 and 1.30) and on a second rotational-aiming task ($r^\star = 0.98$) confirm that the apparatus reproduces this, and serve as calibration rather than as an empirical law. The main result is a **real model swept through that boundary**: a closed-loop Sonnet 4.6 agent, judging every frame itself, clears **32/32** in-window decisions below $r \approx 1$ and **0/26** above it ($r^\star = 1.05$), with nothing varied but the injected delay — the same correct decision succeeds paused and fails late. Past the boundary its *only* surviving episodes are ones that misjudged early, their perception error cancelled by the delay: the two failure axes compensate, which an aggregate score cannot see. On the perception axis, blind single-frame judgment at zero delay scales with model capability (Opus 90%, Sonnet 80%) and collapses to chance under fog. Measuring five real vision models' decision latency against the task's 257 ms window, **all land past the boundary** — hosted frontier models at $r \approx 4$–6, local open VLMs at $r \approx 31$–47 — so none can act inside the window in real time, yet the same decisions succeed when the world is paused. Pausable/turn-based deployment converts a slow-but-accurate VLM from failing to viable, and "VLMs are too slow to play games" is partly an artifact of the time regime they are benchmarked in.

1 Introduction

When a vision-language model plays a real-time game and walks into a pit, the post-mortem is ambiguous. Did it fail to *see* that a jump was needed (perception), or did it see correctly and *act too late* (timing)? Aggregate game benchmarks — score, win rate, levels cleared — fold these two failures into one number, so "the VLM is bad at this game" can mean either, and usually means some unknown mix. The two have opposite fixes: a perception failure wants a better model; a timing failure wants a slower clock. Conflating them is why "VLMs are too slow to play games" is repeated as though it were a single fact.

Our move is isolation. We reduce an episode to *one* correct decision — a first-person pit the player auto-walks toward, where the only choice is *when* to jump — so that on any given run exactly one of the two failures can occur, and we can say which. The apparatus is deterministic and headless, and it injects a control delay D (a decision made at tick t is applied at $t{+}D$), which lets us vary *lateness* completely independently of *judgment quality*: a perfect-judgment oracle still fails if its action lands too late.

The headline result is **S5**: a closed-loop Sonnet 4.6 agent, judging every frame itself, is swept through the boundary by injected delay and clears 32/32 in-window decisions below $r \approx 1$ and 0/26 above it, and its only clears past the boundary are *delay-rescued misjudgments*. Around it:

- **Calibration (S1, S2, S4).** For a policy that fires at the window's leading edge, clearing iff $r < 1$ is arithmetic, not a discovery. Oracle sweeps on the pit (S1; $r^\star = 1.17$, bracketed between 1.04 and 1.30 by the delay grid) and on a rotational aiming task (S4; $r^\star = 0.98$) check that the sim's dynamics, the in-window label, and r reproduce it without leaking tolerance, and the oracle's same-decision flip (S2) previews S5 on a perfect judge.
- **Perception axis.** Blind single-frame judgment at zero delay scales with model capability and collapses to chance under fog — only in the models that read depth.
- **Latency axis.** Five real vision models' decision latencies against the task's 257 ms window: all past the boundary, hosted frontier models at $r \approx 4$–6 and local open VLMs at $r \approx 31$–47.
- **Control (S3).** Action-repeat adds polling delay and shifts the transition to lower explicit r; it is a cost lever, not a latency fix.

2 Related work

Five works are closest; we differentiate from each.

- **OmniGameArena** (Lin et al., 2026, arXiv:2606.09826) — has paused (PDQ) vs. latency-controlled real-time (LCRT) clock modes, but only as **aggregate whole-game** comparisons; no per-decision in-window label, no single-decision flip. We add both, and the r coordinate that makes latency measurements comparable across tasks.
- **VideoGameBench** (Zhang et al., 2025, arXiv:2505.18134) — "Lite" (paused) vs. real-time; names the stale-action mechanism but leaves perception confounded ("it's not just latency"). We resolve exactly that confound by isolating one decision.
- **"Win Fast or Lose Slow"** (Kang et al., NeurIPS 2025, arXiv:2505.19481) — owns the latency–quality tradeoff, but its latency lever is quantization/model-size, which *also* degrades quality, and its ≈200 ms point is an environment action-rate cap, not a task-window threshold. Our sweep varies latency **independently** of the model — the apparatus injects control delay D with the policy fixed.
- **BLINK** (Fu et al., ECCV 2024, arXiv:2404.12390) — canonical "MLLMs fail at depth," but static MCQ. Ours is interactive, parametric, oracle-normalized.
- **Ramstedt & Pal, "Real-Time RL"** (NeurIPS 2019, arXiv:1911.04448) [+ "At Human Speed," arXiv:1810.07286] — the RL parent of the real-time/delay result (monotone degradation for learned policies). We express the same delay cost as r for *pausable VLM agents*, show a real VLM crossing it, and show turn-based deployment stepping back over it.

We explicitly do **not** claim to be first to study the latency–quality tradeoff (Win Fast) nor first to isolate timing from competence in the aggregate (Real-Time Deadlines, arXiv:2601.13206; OmniGameArena). Our claim is the single-decision dissociation on a real model, the compensating-axes observation past the boundary, and the quality-controlled r coordinate that makes the latency measurements interpretable.

3 Task and apparatus

A first-person raycaster. The player auto-walks toward a pit; the only decision is *when* to jump. Knobs: pit width (sets the action-window W), fog/visibility (perception), approach speed v, seed. The sim is deterministic and headless; control delay D (ticks) is **injected** — a decision made at tick t is applied at $t{+}D$ — so D is an independent variable decoupled from wall-clock. Real per-model latency L is measured separately (timed calls) and mapped to the r axis via L/tick. An oracle policy (true pit distance) is the perfect-judgment ceiling.

Ground-truth label. `in_window` = whether a delay-0 takeoff at the decision position clears, computed from the sim's *own* dynamics (not an analytic approximation), so `in_window` exactly predicts `cleared` at $D{=}0$. The action window is $W = w_{hi} - w_{lo}$ with w_{lo}, w_{hi} derived by simulating takeoffs; for pit widths beyond the jump reach the window is empty ($W \le 0$, unclearable) and those configs are excluded from fits (reported, not silently dropped).

$$r = \frac{v \cdot D}{W} \text{ (oracle sweep)} \qquad \text{or} \qquad r = \frac{v \cdot (L/\text{tick})}{W} \text{ (real-model placement).}$$

What r does and does not tell you. A policy that commits at the window's leading edge w_{lo} has its delayed takeoff land at $w_{lo} + vD$, which clears iff $vD < W$, i.e. iff $r < 1$. The location of the boundary is therefore fixed by where the policy fires, not discovered by sweeping; a policy that fired at the window's center would place it at $r = 0.5$. r is the coordinate in which that statement is exact and task-independent, which is what makes a measured latency L interpretable against any window. The oracle sweeps in §4.1–4.3 are calibration: they check that the dynamics, the label, and r reproduce the boundary with no leaked tolerance. The question with empirical content — where a *real* judge's commit point falls, and whether its outcome flips at that boundary with nothing else varied — is §4.4.

4 Results

4.1 Calibration: the oracle boundary (S1)

The oracle fires at the first in-window position, so a delayed takeoff lands at $w_{lo} + vD$ and clears iff $r < 1$; the sweep's job is to confirm that the apparatus reproduces this. It does. The default grid (4 pit widths $\times$ 7 delays, $D \in \{0, 4, \ldots, 24\}$) has 28 unique configurations. The 1680 committed rows replicate each 60$\times$ across fog and seed, which the oracle ignores (every replicate is identical), and the 7 configurations at pit width 4 have an empty window ($W \le 0$, unclearable) and are excluded. On the remaining 21 configurations the pooled logistic fit gives $r^\star = 1.17$. Because the delay grid is discrete, the data bracket the edge rather than pinpoint it: the last clear is at $r = 1.04$ and the first failure at $r = 1.30$, both at pit width 2, and 1.17 is the logistic midpoint of that gap. The other widths do not straddle the edge within this grid — width 1 ($W = 2.08$) never exceeds $r = 0.81$ and always clears, and width 3 ($W = 0.08$) is already at $r = 3.5$ at the first nonzero delay and

always fails — so the edge is located at one width and the others are consistent with it. The clear at $r = 1.04$ is physical: a jump that fires on the same tick can take off from just past the near edge, extending tolerance a fraction of a tick beyond the nominal window. The transition is a step because the oracle is deterministic.

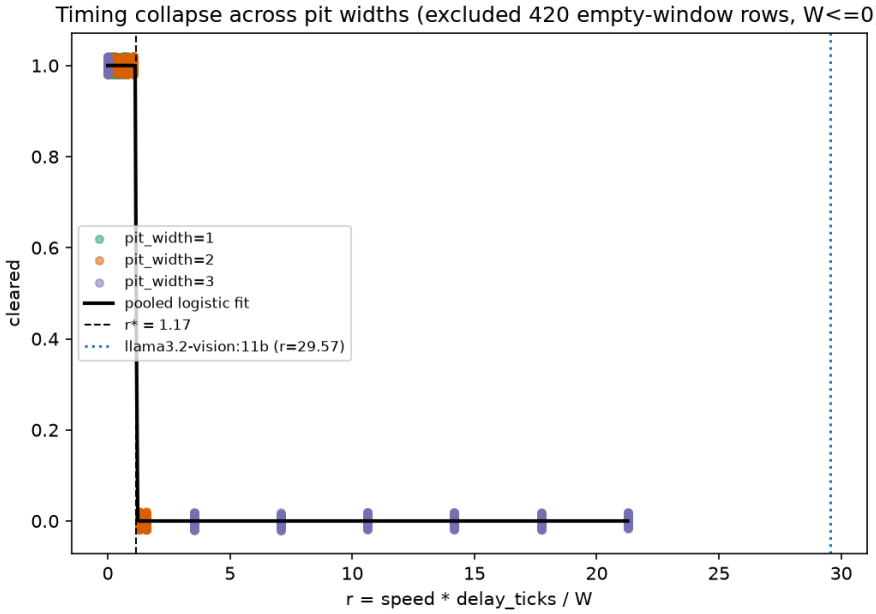


Figure 1: S1 (calibration). 21 unique oracle configurations (the plotted rows replicate each 60× across fog/seed); pooled logistic $r^\star = 1.17$, bracketed between $r = 1.04$ and 1.30 at pit width 2.

4.2 Same-decision flip on the oracle (S2)

Restricting to oracle-correct decisions (`in_window` = 1; the oracle's commit position does not depend on D), the **identical** decision clears 3/3 unique configurations at $D = 0$ and 0/8 once $r \geq 1.30$. The one in-window decision past $r = 1$ that clears is the $r = 1.04$ same-tick case above. Only execution timing changed. We do not summarize this as a pooled probability: six of the 9 configurations with $r > 1$ are at pit width 3, where r runs from 3.5 to 21, so a rate over them would describe the delay grid rather than the task. The flip on a real judge, with decisions on both sides of the edge, is S5.

4.3 The same calibration on a second task (S4)

Is the apparatus's behaviour specific to jump-timing? We built a second task in a different modality: a **sweeping-aim** task. The player rotates in place at constant angular speed ω; a target subtends an angular window $[\theta - h, \theta + h]$ (width $2h$); the single decision is *fire*, and a control delay D rotates the aim ωD further before the shot lands — overshoot if it exceeds the window. By construction the dimensionless cost is $r = \omega D/(2h)$, the exact angular analog of the pit's vD/W. The aim episode is pure kinematics and emits the same schema, so the *identical* fit machinery applies.

Because the aim oracle also fires at the leading edge, a hit iff $r < 1$ is again arithmetic; the test is whether an independent implementation of the kinematics and the label reproduces it. It does: across 205 unique configurations at five target widths (4.6°–13.8°; one seed, the aim oracle being

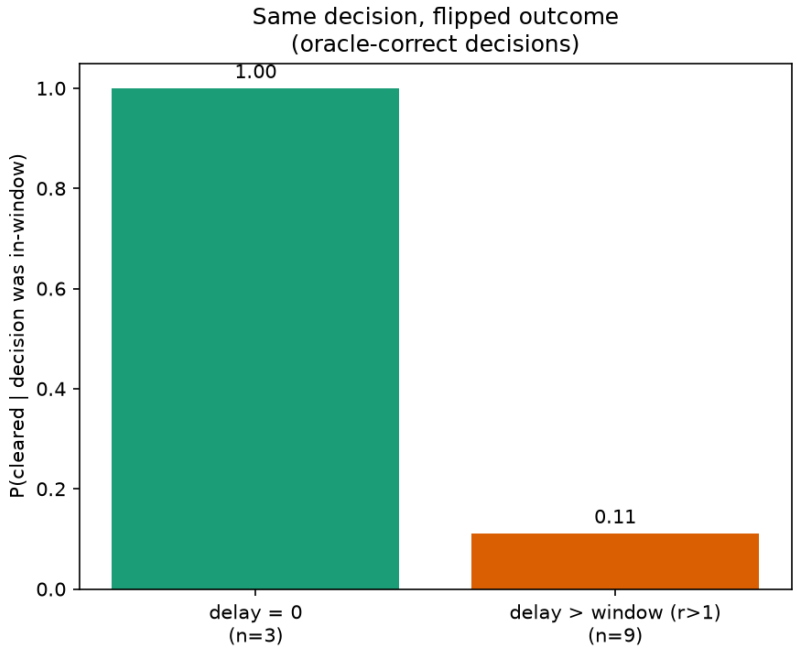


Figure 2: S2. Oracle-correct decisions: 3/3 clear at $D = 0$; of 9 unique configurations past $r = 1$, only the $r = 1.04$ same-tick case clears (0/8 at $r \geq 1.30$).

deterministic) the pooled logistic fit gives $r^\star = 0.98$, and at every width the last hit lies at $r = 0.87$–0.96 and the first miss at exactly $r = 1.00$. The offset from the pit's 1.17 is the oracles' firing phase: the pit's takeoff can extend just *past* the near edge (a hair above 1), while the aim fires just *inside* the near edge, forfeiting a sub-tick of tolerance (a hair below 1). Two tasks agreeing on $r \approx 1$ shows that r is the right coordinate for both and that neither implementation leaks tolerance. It is not evidence of an empirical law: a policy that fired at the window's center would place its boundary at $r = 0.5$ in the same apparatus. What the next section asks is where a real judge's commit point actually falls.

4.4 A real model through the boundary (S5)

The oracle sections calibrate the boundary; here a real model is swept *through* it with its own judgment in the loop. Because the apparatus decouples sim time from wall-clock, the model's (slow, variable) API latency is free, and r is engineered exactly as in S1: Sonnet 4.6 judges *every grounded frame* of a closed-loop blocking episode (pit width 2, $v = 0.07$, fog 0, seeds 1–6), its committed jump is applied D ticks later, and sweeping $D \in \{0, \ldots, 31\}$ sweeps $r = vD/W$ from 0 to 2, densely around $r = 1$ (72 episodes; 6,612 vision calls).

Judge protocol. The episode prompt used for the latency battery (§4.6) turns out never to elicit JUMP near the window from *any* hosted model: it describes the pit as "the black band across the floor ahead," which stops matching the close-range view — inside the window the pit is a black region filling the bottom of the frame, and the visible checkered floor is the pit's *far* side. The S5 judge instead answers a literal perceptual question ("what touches the bottom border of the image: tiles or black?") at high reasoning effort, with a commit rule of two consecutive JUMP answers. The commit rule matters: in prompt-development batteries we measured isolated false

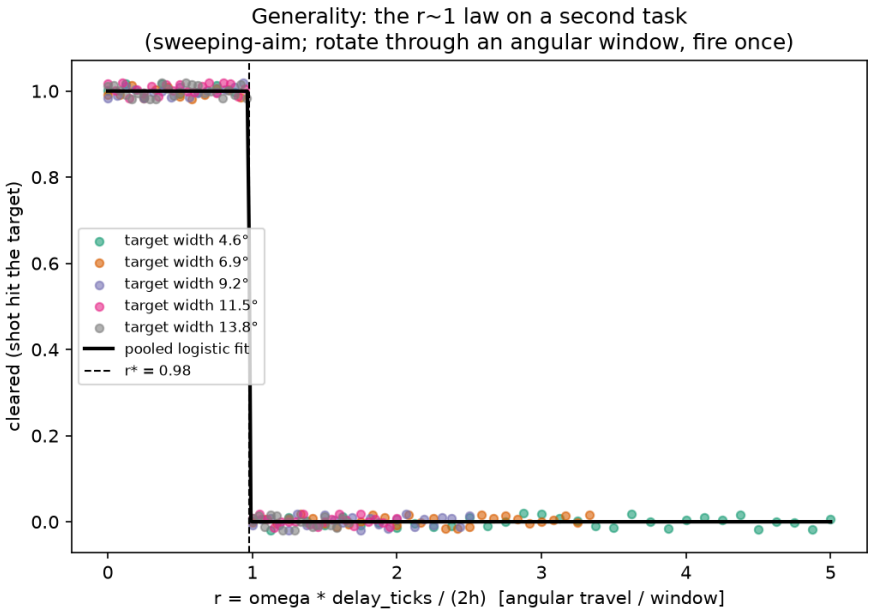


Figure 3: S4 (calibration on a second task). The rotational aiming oracle reproduces the $r \approx 1$ boundary ($r^\star = 0.98$; 205 configurations, 5 target widths).

JUMPs at the 1–2% per-call level far from the pit, and any such rate fires almost surely somewhere over the ∼90 consulted approach frames — a first-passage failure mode that single-frame accuracy batteries cannot see. We validated the final judge on all 108 approach frames × 3 samples: zero far-field JUMPs, unanimous JUMP inside the window, and isolated misfires only in the last half-tile before the window ($x = 8.10$–8.24) — the source of the 9/72 early commits below. This judge was developed against, and validated on, the same renderer, pit width, and fog level the sweep uses; it is a judge tuned to this scene, and we make no claim that it transfers. The bottom-edge cue is nearly ideal in this renderer: the pit's black region first touches the frame's bottom edge at $x = 9.01$, one tick after the window opens ($w_{lo} = 8.92$).

The boundary holds for a real judge. Episodes whose decision lands in the window clear **32/32 through** $r = 0.97$, 4/5 at $r = 1.04$, and **0/26 from** $r = 1.10$ **on**; the same logistic fit as S1 gives $r^\star = 1.05$, inside the oracle's bracketing gap $[1.04, 1.30]$ and at its last-clear edge (the same-tick takeoff tolerance of S1). The transition is nearly as sharp as the oracle's, for an unplanned reason: the model's commit position is almost deterministic — 51/72 episodes commit at $x = 8.94$ and 12 at $x = 9.01$, straddling the cue's pixel-level onset (the model begins answering "black" one tick before the black region literally reaches the bottom edge at $x = 9.01$). The anticipated noisy psychometric curve compresses to a near-step; this judge's decision-position variance is sub-tick, so the model behaves as a *slightly-early oracle* rather than a noisy one. Given that commit point, the *location* of $r^\star$ follows from the kinematics exactly as in S1. What S5 establishes that the oracle cannot is the dissociation itself on a real judge: the same model, making the same frame-by-frame decisions, clears every in-window decision below the boundary and none above it, with nothing varied but D.

Delay-rescued misjudgments. 9/72 episodes commit early ($x = 8.10$–8.24, out of window — a perception error; these are excluded from the timing fit above). At $D \leq 8$ every such episode fails, landing short. In the band $D = 15$–24 ($r = 0.97$–1.56), *every* early episode **clears** (5/5): the delay pushes the too-early takeoff into the window. Past $r \approx 1.1$ the *only* surviving episodes are the misjudged ones; by $r = 2.01$ the delay overshoots even the early takeoff and the rescue band closes. The two failure axes do not merely dissociate — on opposite sides of the boundary they *compensate*, which is only visible because the apparatus labels each decision in- or out-of-window.

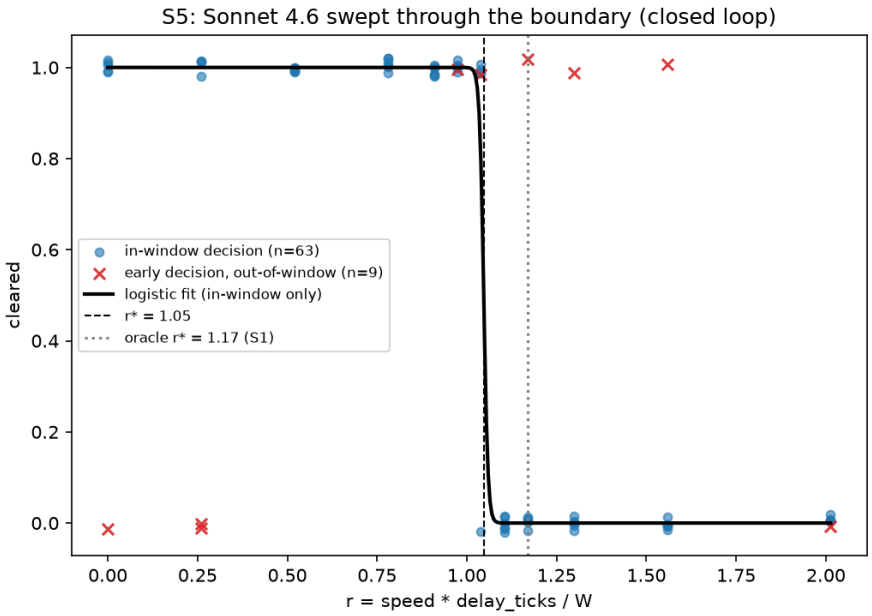


Figure 4: S5. Sonnet 4.6 swept through the boundary in closed loop ($r^\star = 1.05$, in-window decisions). Crosses are out-of-window (early) decisions: they fail below the boundary and are the only survivors past it (delay-rescued perception errors).

4.5 Perception axis

A capable vision model *can* judge the jump from a single frame, and fog collapses it. We rendered standing decision frames across approach positions at two visibility levels and had **four blind vision models** — three Claude models (Haiku 4.5, Sonnet, Opus) and one local open VLM (`qwen2.5-vl:7b` via Ollama) — each shown only the image (no coordinates, window, or labels; identical seed-42 shuffled frame set) call JUMP or WAIT on each; we score each call against the ground-truth window. Judgment accuracy at zero delay ($n = 10$ frames per model per condition):

Two things fall out. **(1) With a clear depth cue, judgment scales with model capability** — Opus 90%, Sonnet 80%, Qwen 60%, Haiku 40%. **(2) The fog collapse appears only in the models that actually read depth.** Opus and Sonnet drop to chance under fog, and each **missed every in-window frame** (0/5): they reported no distinguishable pit band even up close, never committed, and would fall. The two weak judges show **no fog effect at all** — but for opposite-looking reasons that are really the same reason: Haiku sits at floor (it fell back on a frame-order heuristic the shuffle nullifies), and Qwen is strongly *JUMP-biased* (it answered JUMP on 18/20 frames, clearing all in-window frames but also most out-of-window ones). Neither is using the depth cue in the first place, so neither has any perception to lose when fog removes it. The fog

Table 1: Blind single-frame judgment accuracy at zero delay ($n = 10$ frames per model per condition; exact counts shown).

Model	Clear (fog=0)	Foggy (fog=1)	Fog effect
Opus	**90%** (9/10)	50% (5/10, chance)	**−40** pts
Sonnet	80% (8/10)	50% (5/10, chance)	**−30** pts
Qwen2.5-VL 7B (local)	60% (6/10)	60% (6/10)	0
Haiku 4.5	40% (4/10)	40% (4/10)	0

collapse is a **signature of genuine depth reading**, present exactly where clear-condition accuracy is high. At $n = 10$ frames per cell the per-model contrasts are directional rather than individually significant (Fisher exact, clear vs. fog: Opus $p = 0.14$, Sonnet $p = 0.35$; pooled Opus+Sonnet 17/20 vs. 10/20, $p = 0.04$) — it is the coherence of the pattern across four models, not any single cell, that carries the claim.

This is the perception half of the dissociation: a model that judges well with a clear depth cue drops to chance when fog obscures the pit's distance — **at zero delay**, i.e. independent of the timing axis. Together, perception and timing are separable failure axes on one task. (Caveat: the Claude judges run via a subagent path, not a controlled API benchmark; the local VLM is a real keyless run. Judges are blinded to coordinates and labels but not to the task.)

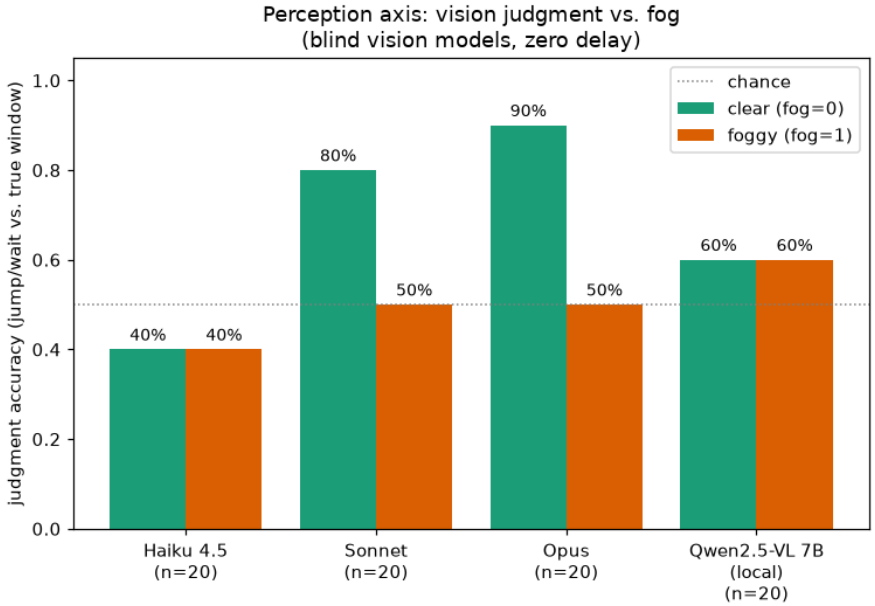


Figure 5: Perception axis. Fog collapses judgment to chance only for the depth-reading models (Opus, Sonnet); JUMP-biased/floor judges show no fog effect.

4.6 Real models' latency against the window

Because the decision cost is exactly $r = L/T_{\text{window}}$, where the window is open for $T_{\text{window}} = (W/v) \cdot \text{tick} = 257\,\text{ms}$ at pit width 2 ($W = 1.078$, $v = 0.07$, tick $= 1/60\,\text{s}$), the $r^\star$ boundary sets a **maximum tolerable decision latency of $r^\star \cdot T_{\text{window}} \approx 300\,\text{ms}$**. We timed five vision models

on the identical 20-frame decision battery, in each model's *fastest* single-call configuration ($n = 20$ each):

Table 2: Measured per-model decision latency (median of $n = 20$ single-frame calls, fastest configuration) against the 257 ms window at pit width 2.

Model	Median L	$r = L/\text{window}$
Haiku 4.5 (hosted API)	0.95 s	≈ 4
Sonnet 4.6 (hosted API)	1.45 s	≈ 6
Opus 4.8 (hosted API)	1.53 s	≈ 6
Qwen2.5-VL 7B (local)	7.9 s	≈ 31
llama3.2-vision 11B (local)	12.1 s	≈ 47

None clear the 300 ms budget. Even the *fastest hosted frontier model* is $\sim 4\times$ too slow, and local open VLMs are 30–47×. This is the concrete version of "VLMs are too slow to play games": the whole measured field would need the world **paused, or slowed 4–47×**, to act inside the window — and the gap is a property of the *time regime*, not the model, since r here is L/window with judgment held out entirely. These are time-to-answer measurements, not judgments: in this fast configuration the hosted models answered WAIT on nearly every frame (Haiku and Sonnet 20/20, Opus 18/20) and the local models answered JUMP on nearly every frame, so the numbers are each model's best case — no extended thinking, and a configuration that actually discriminates (the S5 judge, ~ 5 s/call) is only slower. Each local model's first call is an order of magnitude faster than the rest (0.4–0.6 s against 8–12 s), consistent with a warm cache; the medians are unaffected. Local latency is on-device — a real, reproducible closed-loop number.

A closed-loop episode (blocking mode) confirms the local model is non-viable, though the two failure modes should be separated: `llama3.2-vision` also *fails perception* — it answered JUMP on all 20 frames (no depth discrimination), so in blocking mode it jumps at tick 0 from $x \approx 2.6$ (far from the pit at $x = 10$) and falls. The latency result is the axis-independent point: $r > r^\star$ dooms even a model that judges correctly, the moment the world is allowed to run in real time.

4.7 Control: action-repeat does not rescue (S3)

Consulting the policy only every k ticks and holding its last decision is a common cost lever. It adds up to $k-1$ ticks of polling delay on top of the explicit D, so it must shift the transition to *lower* explicit r, and it does. Measured $r^\star$ vs. k at pit width 2 ($D \in \{0, 2, \ldots, 40\}$, one seed): k=1 → 1.10, k=2, 4 → 0.97, k=8, 16 → 0.71; the ties are grid quantization (a 2-tick delay step is 0.13 in r at this width). This is a control, confirming that the apparatus charges polling delay the same as injected delay. Action-repeat is a compute/cost lever, not a latency fix.

5 Discussion

The two failure axes have opposite fixes, and the framing makes the requirement quantitative. A perception failure wants a better model: no amount of extra time helps a judge that reports no distinguishable pit band under fog. A timing failure wants a slower clock: a model that commits at the window's leading edge is viable exactly when $r = L/T_{\text{window}} < r^\star \approx 1$, so a deployment can cross the boundary by pausing the world ($r \to 0$), slowing it (raising T_{window}), or cutting latency — and the measured field needs 4–47× on the latter two. S5 is the direct demonstration: the same

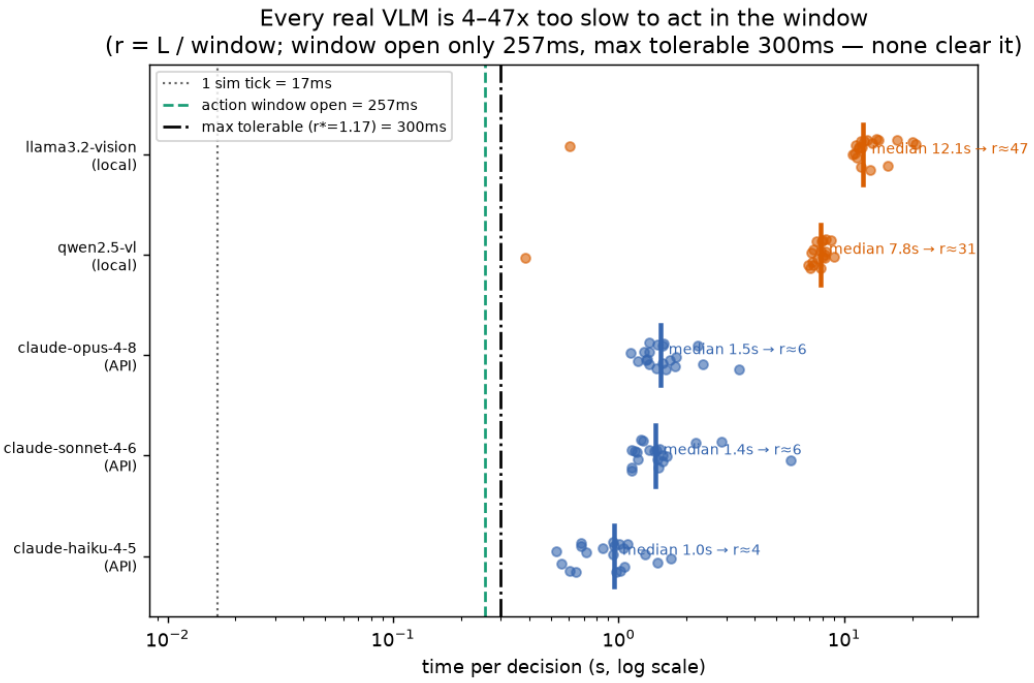


Figure 6: Real per-model decision latency vs. the time budget. Hosted Haiku/Sonnet/Opus ($r \approx 4$–6) and local Qwen/llama3.2-vision ($r \approx 31/47$); all past the ≈300 ms max-tolerable budget.

model, same judgment, clears 32/32 below the boundary and 0/26 above it, with nothing varied but r. The practical reading is that pausable or turn-based deployment converts every slow-but-accurate model we measured from failing to viable, with no change to the model; conversely, a real-time benchmark score is a product of judgment *and* time regime, and "VLMs are too slow to play games" is partly a fact about the clock they are handed. S5's delay-rescued misjudgments add a sharper corollary: past the boundary, scores can even *reward* bad perception — the axes compensate, and an aggregate benchmark cannot tell.

6 Limitations

- **The real-model sweep (S5) is one model at one pit width, at fog 0.** Its judge uses a revised prompt (the latency-battery prompt elicits no JUMPs near the window from any hosted model) plus a two-consecutive-JUMP commit rule, developed against and validated on the same 108 approach frames the sweep draws from — a judge tuned to this scene, and a documented apparatus change, not the shipped default. Its r axis is engineered by injected delay with the model's judgment in the loop; the model's own wall-clock latency (∼5 s/call at high effort) remains far past $r^\star$, so a wall-clock-real sweep is still impossible for the measured field — that is the latency section's point, not a gap. The anticipated *noisy* psychometric curve did not materialize — this judge's decision variance is sub-tick — so mapping a genuinely noisy transition needs a weaker or more stochastic judge.
- **The oracle calibration has 21 informative configurations and brackets the edge at one pit width.** Within the default delay grid, width 1 never reaches the edge and width 3 is past it at the first nonzero delay; only width 2 has clears and failures on both sides. The 1680-

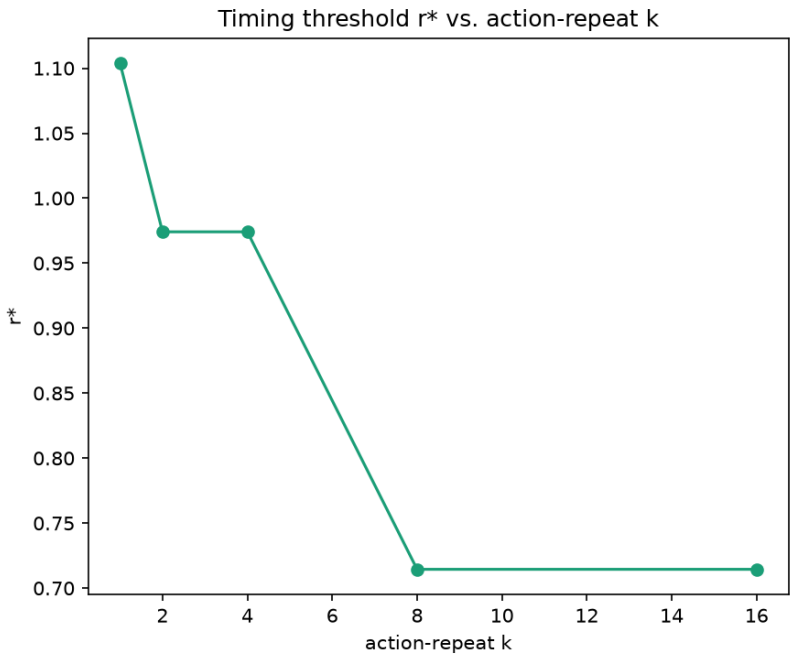


Figure 7: S3 (control). Action-repeat does not rescue the transition; larger k fails at lower explicit r.

row file is 60× replication over fog and seed that the oracle ignores. A finer, width-adapted delay grid would bracket the edge independently at each width.

- **The location $r \approx 1$ is definitional for leading-edge policies.** The agreement between the pit and aiming oracles (S1, S4) checks the apparatus; it is not evidence that real agents on other tasks share a boundary. Whether real judges on richer, multi-decision tasks have a stable commit point — the quantity that would make r predictive for them — is untested; S5 shows one judge on one scene does.
- **The perception axis** mixes three Claude judges (run via a subagent path, not a controlled API benchmark) with one real keyless local VLM (`qwen2.5-vl:7b`), at $n = 10$ frames per condition — large enough for the cross-model pattern, too small for per-model significance (Fisher exact $p = 0.14$–0.35). The open VLMs are weak/biased judges — Qwen is JUMP-biased (60% clear, no fog effect) and `llama3.2-vision:11b` is always-JUMP (contributes latency only, not a perception point). A controlled, latency-timed multi-provider API sweep still needs a key.
- **The latency battery measures time-to-answer, not judgment.** In the fast configuration the hosted models answered WAIT on nearly every frame; a configuration that discriminates is slower, so the measured r values are lower bounds. Latencies are network- and hardware-dependent, but they are real decision latencies, not stand-ins.
- **No human baseline** — the oracle stands in as the perfect-judgment ceiling. Human psychophysics is future work.

7 Future work

Extending the S5 real-model sweep to more models, pit widths, and fog levels (and to a judge noisy enough to trace a genuinely smooth psychometric curve); a width-adapted oracle grid that brackets the edge at every width; a multi-provider latency panel (Gemini/GPT + more open VLMs); a human psychophysics arm; and multi-decision tasks, to test whether real judges have a stable enough commit point for r to stay predictive beyond single reflexes.

Reproducibility

Code, data, and figure-generating scripts are at `https://github.com/star7js/judge-vs-react`, under `eval/pit/` (analysis and sweeps) and `raycaster/` (the deterministic sim, including `--task aim` for the second task). The perception judgments (`perception_judgments.jsonl`, 80 rows across 4 models), per-decision latencies (`vlm_latency.jsonl`, 100 rows across 5 models), the oracle and aim sweeps, and the S5 real-model sweep (`eval/pit/realmodel/`: driver, revised judge, 108-frame battery validation, and the 72-episode JSONL) are committed. The local-VLM paths (Ollama) and the Claude subagent judges run without an API key; the hosted-API latency numbers and the S5 sweep required one.

References

[1] M. Lin et al. *OmniGameArena: A Unified UE5 Benchmark for VLM Game Agents with Improvement Dynamics*. arXiv:2606.09826, 2026.

[2] A. L. Zhang, T. L. Griffiths, K. Narasimhan, O. Press. *VideoGameBench: Can Vision-Language Models complete popular video games?* arXiv:2505.18134, 2025.

[3] H. Kang et al. *Win Fast or Lose Slow: Balancing Speed and Accuracy in Latency-Sensitive Decisions of LLMs*. NeurIPS 2025, arXiv:2505.19481.

[4] X. Fu et al. *BLINK: Multimodal Large Language Models Can See but Not Perceive*. ECCV 2024, arXiv:2404.12390.

[5] S. Ramstedt, C. Pal. *Real-Time Reinforcement Learning*. NeurIPS 2019, arXiv:1911.04448.

[6] V. Firoiu, T. Ju, J. Tenenbaum. *At Human Speed: Deep Reinforcement Learning with Action Delay*. arXiv:1810.07286, 2018.

[7] N. K. R. Sehgal, S. C. Guntuku, L. Ungar. *Real-Time Deadlines Reveal Temporal Awareness Failures in LLM Strategic Dialogues*. arXiv:2601.13206, 2026.